Advanced (DE-focused) SQL PLP Assignment

Q1) Topic: DML: INSERT, UPDATE & DELETE

Using the e-commerce tables (customers, orders, products, order_items), write the following DML statements:

a) INSERT a new customer. Write an INSERT adding 3 order_items rows in a single statement.

Query Query History

```
1  INSERT INTO customers
2  (id,name, city, email, registration_date)
3  VALUES
4  (8,'Ananya', 'Hyderabad', 'ananya@gmail.com', '2026-11-10');
```

Data Output Messages Notifications

```
INSERT 0 1

Query returned successfully in 72 msec.
```

Query Query History

```
1  INSERT INTO order_items
2  (order_id, product_id, quantity)
3  VALUES
4  (15, 4, 2),
5  (15, 5, 1),
6  (16, 11, 2);
```

Data Output Messages Notifications

```
INSERT 0 3

Query returned successfully in 73 msec.
```

b) UPDATE all Electronics products: increase price by 10%. UPDATE to set stock_qty=0 for products never ordered (use subquery).

Query Query History

```
1  UPDATE products
2  SET price = price * 1.10
3  WHERE category = 'Electronics';
```

Data Output Messages Notifications

```
UPDATE 8

Query returned successfully in 63 msec.
```

Query Query History

```sql
UPDATE products p
SET stock_qty = 0
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.id
);
```

Data Output Messages Notifications

```
UPDATE 4

Query returned successfully in 70 msec.
```

c) DELETE all orders placed before 2020 where total_amount < 100. Explain why you should test with SELECT first before running DELETE.

Query Query History

```sql
SELECT *
FROM orders
WHERE order_date < '2020-01-01'
AND total_amount < 100;
```

Data Output Messages Notifications

SQL

id [PK] integer	customer_id integer	order_date date	total_amount numeric (10,2)

Query Query History

```sql
DELETE FROM orders
WHERE order_date < '2020-01-01'
AND total_amount < 100;
```

Execute SELECT before DELETE:

1. Confirms which rows will be deleted.
2. Prevents accidental deletion of unwanted records.
3. Helps validate the `WHERE` condition.
4. Reduces the risk of permanent data loss.

d) Write a transaction: insert new order, insert 2 order_items, update customer's last_order_date -- all atomically.

Query | Query History

```
BEGIN;
-- 1. Insert a new order
INSERT INTO orders
(order_id, customer_id, order_date, total_amount)
SELECT 17,7,CURRENT_DATE,SUM(p.price * v.quantity)
FROM (VALUES (1, 2), (2, 1)) AS v(product_id, quantity)
JOIN products p ON p.product_id = v.product_id;

-- 2. Insert two order_items
INSERT INTO order_items(order_id, product_id, quantity)
VALUES(17, 1, 2),(17, 2, 1);

-- 3. Update customer's last_order_date
UPDATE customers
SET last_order_date = CURRENT_DATE
WHERE customer_id = 7;

COMMIT;
```

Data Output | Messages | Notifications

```
COMMIT

Query returned successfully in 75 msec.
```

e) Predict the result: -- products: (1,'Phone',100.00,'Electronics'),(2,'Shirt',200.00,'Clothing')

```
UPDATE products
SET price=price*1.10
WHERE category='Electronics';

SELECT id, price FROM products ORDER BY id;
```

Final result set:

id	price
1	110.00
2	200.00

Q2) MERGE / UPSERT

You receive a daily file of updated customer records. Write a MERGE / UPSERT statement:

a) MERGE: if customer exists (matched on customer_id) -- UPDATE name, email, city; if new -- INSERT all columns.

Query | Query History

```sql
MERGE INTO customers AS target
USING (VALUES(1, 'Aarav Sharma', 'Pune','aarav.new@gmail.com', DATE '2026-11-05'),
        (3, 'Ishaan Kumar', 'Hyderabad','ishaan.new@gmail.com', DATE '2026-11-05'),
        (8, 'Ananya', 'Mumbai','ananya@gmail.com', DATE '2026-11-05')
) AS source(customer_id,customer_name,city,email,last_order_date)
ON target.customer_id = source.customer_id

WHEN MATCHED THEN
    UPDATE SET customer_name = source.customer_name,email = source.email,city = source.city

WHEN NOT MATCHED THEN
    INSERT (customer_id,customer_name,city,email,last_order_date)
    VALUES (source.customer_id,source.customer_name,source.city,source.email, source.last_order_date);
```

Data Output | Messages | Notifications

```
MERGE 3

Query returned successfully in 53 msec.
```

b) Rewrite using PostgreSQL INSERT ... ON CONFLICT DO UPDATE (UPSERT). Explain the difference between MERGE and ON CONFLICT.

Query | Query History

```sql
INSERT INTO customers (customer_id,customer_name,city,email,last_order_date)
VALUES
    (1, 'Aarav Sharma', 'Pune',
        'aarav.new@gmail.com', DATE '2026-11-05'),
    (3, 'Ishaan Kumar', 'Hyderabad',
        'ishaan.new@gmail.com', DATE '2026-11-05'),
    (8, 'Ananya', 'Mumbai',
        'ananya@gmail.com', DATE '2026-11-05')

ON CONFLICT (customer_id)
DO UPDATE SET
    customer_name = EXCLUDED.customer_name,
    city = EXCLUDED.city,
    email = EXCLUDED.email;
```

Data Output | Messages | Notifications

```
INSERT 0 3

Query returned successfully in 58 msec.
```

c) MERGE implementing SCD Type 2: expire old record (set effective_to, is_current=false) and INSERT new version for city changes. Insert normally for new customers.

Query | Query History

```sql
MERGE INTO customer_history AS target
USING (VALUES(1, 'Alice', 'alice@gmail.com', 'Pune'),
    (3, 'Carol', 'carol@gmail.com', 'Chennai')) AS source(customer_id, customer_name, email, city)

ON target.customer_id = source.customer_id
AND target.is_current = TRUE

WHEN MATCHED AND target.city <> source.city THEN
    UPDATE SET
        effective_to = CURRENT_DATE,
        is_current = FALSE

WHEN NOT MATCHED THEN
    INSERT (customer_id, customer_name, email, city,effective_from, effective_to, is_current)
    VALUES (source.customer_id, source.customer_name,source.email, source.city,CURRENT_DATE, NULL,TRUE);
```

Data Output | Messages | Notifications

```
MERGE 1

Query returned successfully in 59 msec.
```

d) Explain when MERGE can cause performance issues on large tables and what alternatives exist (staging tables + DELETE + INSERT pattern).

MERGE can cause performance issues when processing large datasets due to:

1. **Full table scans:** Searching for matching records can take considerable time.
2. **Locking:** Concurrent transactions may have to wait while records are being updated.
3. **High memory usage:** Processing large datasets requires more system resources.
4. **Index maintenance:** Updating indexed columns adds processing overhead.
5. **Large transactions:** Processing many records in one transaction can affect performance.

Alternatives to improve MERGE performance

1. Create indexes on matching columns

Create indexes on columns used in the `ON` condition, such as `customer_id`. This helps the database find matching records faster instead of scanning the entire table.

2. Process records in smaller batches

Instead of processing millions of records in one operation, divide them into smaller groups (e.g., 10,000 records per batch). This reduces memory usage, transaction size and locking time.

3. Use separate UPDATE and INSERT statements

Perform updates and insertions independently:

- `UPDATE` modifies records that already exist.
- `INSERT ... WHERE NOT EXISTS` inserts only new records.
  This can be more efficient when the operations are simpler than a complex MERGE.

4. Use staging tables

Load incoming data into a staging table first. Clean, validate and remove duplicate records before updating the main table. This is useful when processing large files or daily data imports.

5. Use INSERT ... ON CONFLICT

For simple UPSERT operations, PostgreSQL's `INSERT ... ON CONFLICT DO UPDATE` can be used instead of MERGE. It handles duplicate-key conflicts by updating existing records and is suitable when only insert-or-update logic is required.

Key point: The best approach depends on the dataset size, indexes, number of operations and database workload.

e) Predict the result: -- target customers: (1,'Alice','Delhi'), (2,'Bob','Mumbai') -- source: (1,'Alice','Pune'), (3,'Carol','Chennai') -- After MERGE (update city for match, insert for new):

```
SELECT id,name,city
FROM customers
ORDER BY id;
```

Final result set:

customer_id	customer_name	city
1	Alice	Pune
2	Bob	Mumbai
3	Carol	Chennai

Q3) Topic: Table Partitioning: Range, List & Hash

You have an orders table with 500 million rows. Implement partitioning:

a) CREATE TABLE for a partitioned orders table using RANGE partitioning on order_date. Create partitions for 2023, 2024, 2025.

Query | Query History

```sql
CREATE TABLE orders_2023
PARTITION OF orders_partitioned
FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');

CREATE TABLE orders_2024
PARTITION OF orders_partitioned
FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');

CREATE TABLE orders_2025
PARTITION OF orders_partitioned
FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 92 msec.
```

b) CREATE TABLE for customers partitioned by LIST on region ('North','South','East','West'). Add a DEFAULT partition for unknown regions.

Query | Query History

```sql
CREATE TABLE customers_partitioned (
    customer_id INT,
    customer_name VARCHAR(100),
    region VARCHAR(20)
)
PARTITION BY LIST (region);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 55 msec.
```

Query | Query History

```sql
CREATE TABLE customers_north
PARTITION OF customers_partitioned
FOR VALUES IN ('North');

CREATE TABLE customers_south
PARTITION OF customers_partitioned
FOR VALUES IN ('South');

CREATE TABLE customers_east
PARTITION OF customers_partitioned
FOR VALUES IN ('East');

CREATE TABLE customers_west
PARTITION OF customers_partitioned
FOR VALUES IN ('West');

CREATE TABLE customers_default
PARTITION OF customers_partitioned
DEFAULT;
```

Data Output Messages Notifications

```
CREATE TABLE

Query returned successfully in 62 msec.
```

c) CREATE TABLE for order_items partitioned by HASH on order_id into 4 buckets. Explain when hash partitioning is better than range or list.

Query Query History

```sql
CREATE TABLE order_items_partitioned (
    order_id INT,
    product_id INT,
    quantity INT
)
PARTITION BY HASH (order_id);
```

Data Output Messages Notifications

```
CREATE TABLE

Query returned successfully in 64 msec.
```

Query Query History

```sql
CREATE TABLE order_items_0
PARTITION OF order_items_partitioned
FOR VALUES WITH (MODULUS 4, REMAINDER 0);

CREATE TABLE order_items_1
PARTITION OF order_items_partitioned
FOR VALUES WITH (MODULUS 4, REMAINDER 1);

CREATE TABLE order_items_2
PARTITION OF order_items_partitioned
FOR VALUES WITH (MODULUS 4, REMAINDER 2);

CREATE TABLE order_items_3
PARTITION OF order_items_partitioned
FOR VALUES WITH (MODULUS 4, REMAINDER 3);
```

Data Output Messages Notifications

```
CREATE TABLE

Query returned successfully in 58 msec.
```

HASH partitioning better because:

1. When data needs to be distributed relatively evenly across partitions.
2. When there is no natural range or category-based grouping.
3. When queries frequently filter using the partition key (`order_id`).
4. It helps avoid uneven partition sizes caused by skewed data values.

d) Show how partitioning improves performance: write a query with WHERE order_date>'2024-01-01' and explain 'partition pruning' and why it reduces I/O.

Query Query History

```sql
SELECT *
FROM orders_partitioned
WHERE order_date > '2024-01-01';
```

Partition pruning is the process of skipping partitions that cannot contain rows matching the query condition.

For this query:

1. The 2023 partition is skipped because its dates cannot satisfy the condition.
2. The 2024 and 2025 partitions are scanned.
3. PostgreSQL does not need to scan all 500 million rows.

Performance:

1. Reduces the number of partitions scanned.
2. Reduces disk I/O by skipping unnecessary partitions.
3. Reduces query execution time.
4. Improves efficiency when working with large datasets.

e) Predict which partition each row goes to:

```sql
CREATE TABLE orders(id INT, order_date DATE)
PARTITION BY RANGE(order_date);

CREATE TABLE orders_2023
PARTITION OF orders FOR VALUES FROM('2023-01-01') TO ('2024-01-01');

CREATE TABLE orders_2024
PARTITION OF orders FOR VALUES FROM('2024-01-01') TO ('2025-01-01');

INSERT INTO orders VALUES(1,'2023-06-15'),(2,'2024-03-20');

SELECT tableoid::regclass, id FROM orders ORDER BY id;
```

Output:

tableoid	id
orders_2023	1
orders_2024	2

Explanation:

1. Row 1 (`2023-06-15`) belongs to `orders_2023` .
2. Row 2 (`2024-03-20`) belongs to `orders_2024` .

> **Note**
>
> `tableoid::regclass` displays the name of the actual partition where each row is stored.

Q4) Topic: Bulk Loading: COPY Command & Performance

You need to load 10 million rows from a CSV file into PostgreSQL efficiently:

a) Write the COPY command to load orders_2024.csv (header row, comma-separated). Show both server-side COPY and client-side \copy versions and explain the difference.

Server-side COPY:

```
COPY orders
FROM '/tmp/orders_2024.csv'
WITH (
    FORMAT CSV,
    HEADER TRUE,
    DELIMITER ','
);
```

Client-side \copy:

```
\copy orders FROM 'C:/data/orders_2024.csv' WITH (FORMAT CSV, HEADER TRUE,
DELIMITER ',');
```

COPY	\copy
Server-side command.	Client-side psql command.
Reads the file from the PostgreSQL server's filesystem.	Reads the file from the client's filesystem.

COPY	\copy
Requires appropriate server file permissions.	Uses the permissions of the client user.
Executed as SQL.	Executed in the `psql` terminal.

> **Note**
>
> `\copy` is a psql command, so it cannot be executed directly in pgAdmin's Query Tool. Use server-side `COPY` in pgAdmin.

b) Load only specific columns (customer_id, order_date, total_amount) from the CSV, skipping the rest.

```
COPY orders (customer_id, order_date, total_amount)
FROM '/tmp/orders_2024.csv'
WITH (
    FORMAT CSV,
    HEADER TRUE,
    DELIMITER ','
);
```

Explanation:

1. Only the specified columns are loaded.
2. Other table columns receive their default values or `NULL` if allowed.
3. The CSV must have its fields arranged in the same order as the specified column

c) List 5 best practices for bulk loading performance (e.g., disabling indexes, UNLOGGED tables, TRUNCATE before load).

5 best practices for bulk loading performance:

1. **Disable indexes before loading:**
   Indexes require additional maintenance whenever rows are inserted. Dropping unnecessary indexes before loading and recreating them afterward can improve loading speed.
2. **Use UNLOGGED tables:**
   UNLOGGED tables reduce write-ahead logging (WAL) overhead, which can improve loading speed. However, their data is not crash-safe and is not replicated to standby servers.

3. **TRUNCATE before loading:**
   If the existing table data is no longer required, use `TRUNCATE` to remove all rows quickly before loading new data.

```
TRUNCATE TABLE orders;
```

4. **Use COPY instead of INSERT:**
   `COPY` loads multiple rows efficiently in one operation, avoiding the overhead of executing individual INSERT statements.
5. **Load data in a controlled transaction:**
   Group loading operations into a transaction where appropriate. This avoids unnecessary transaction overhead and allows the load to be committed or rolled back as a unit.

d) After bulk loading, which maintenance commands should you run (ANALYZE, VACUUM, REINDEX) and in what order? Explain what each does.

Order: ANALYZE → VACUUM → REINDEX

1. **ANALYZE**

```
ANALYZE orders;
```

1. Collects statistics about table conte
2. Helps the query planner choose efficient execution pl
3. Particularly important after loading a large amount of d
4. **VACUUM**

```
VACUUM orders;
```

1. Reclaims space occupied by dead tuples.
2. Makes space reusable.
3. Updates visibility information.
4. Not usually necessary immediately after a pure bulk INSERT, because newly inserted rows are not dead tuples.
5. **REINDEX**

```
REINDEX TABLE orders;
```

1. Rebuilds indexes.
2. Can help when indexes are bloated or need rebuilding.
3. Not normally required after every bulk load.

> **Important**
>
> ANALYZE is generally the most important command after bulk loading. VACUUM and REINDEX should be used when necessary rather than automatically after every load.

e) Predict the result:

```
-- orders table is empty before COPY
-- orders_2024.csv contains 3 rows
COPY orders
FROM '/tmp/orders_2024.csv' CSV HEADER;

SELECT COUNT(*), MAX(total_amount), MIN(total_amount) FROM orders;
```

Expected result:

1. `COUNT(*)` = 3
2. `MAX(total_amount)` = Highest `total_amount` in the CSV.
3. `MIN(total_amount)` = Lowest `total_amount` in the CSV.

Q5) Topic: Advanced Indexes: Composite, Partial & Covering

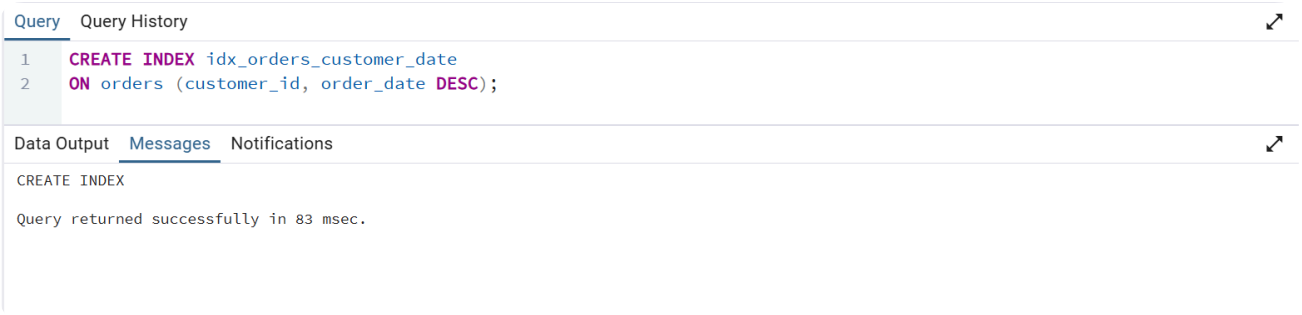
Optimizing a slow reporting query joining orders and customers with multiple column filters:

a) Explain the difference between B-tree, Hash, and GIN indexes. Give one use case where each is the best choice.

Index type	Description	Best use case
B-tree	Stores values in sorted order. Supports equality, range comparisons and sorting.	Searching orders by date or amount.

Index type	Description	Best use case
Hash	Uses a hash function to locate values. Supports equality comparisons only.	Exact-match searches such as `customer_id = 42`.
GIN	Stores entries for individual elements within composite values. Useful for searching inside arrays, JSONB and text-search documents.	Searching JSONB fields or array elements.

b) Create a COMPOSITE index on orders(customer_id, order_date DESC). Explain the column order rule and what 'left-prefix' means.



Column order rule:

1. Place columns frequently used in equality conditions first.
2. Place columns used in range conditions or sorting afterward.
3. Choose the order based on the actual query patterns.

Left-prefix rule:
A composite index on `(customer_id, order_date)` can efficiently support searches using:

1. `customer_id`
2. `customer_id` and `order_date`

But it generally cannot efficiently support searches using only `order_date`.
This is because `customer_id` is the leftmost column.

c) Create a PARTIAL index on orders(total_amount) WHERE status='completed'. Explain when partial is better than full and how much storage it saves.

Query Query History

```sql
CREATE INDEX idx_orders_completed
ON orders (total_amount)
WHERE status = 'completed';
```

Partial index better than a full index when:

1. Queries frequently access only a specific subset of rows.
2. The indexed condition selects a small portion of the table.
3. It avoids indexing rows that are not relevant to those queries.

Storage saved:
Storage savings depend on the proportion of rows satisfying the condition.
For example, if only 20% of orders are completed, the index contains approximately 20% of the rows that a full index would contain. Actual storage savings are not necessarily exactly 80%, because index overhead also matters.

d) Create a COVERING index including all columns for: SELECT order_date, total_amount FROM orders WHERE customer_id=42. Explain 'index-only scan'.

Query Query History

```sql
CREATE INDEX idx_orders_covering
ON orders (customer_id)
INCLUDE (order_date, total_amount);
```

Data Output Messages Notifications

```
CREATE INDEX

Query returned successfully in 59 msec.
```

Explanation:

1. `customer_id` is the search column.
2. `order_date` and `total_amount` are included in the index.
3. PostgreSQL can retrieve all required columns from the index without needing to fetch the corresponding table rows, when visibility information permits.

Index-only scan:
An index-only scan retrieves the required data directly from the index rather than accessing the table for every matching row.
Benefits:

1. Reduces table access.
2. Reduces disk I/O.
3. Can improve query performance.

e) Predict what EXPLAIN will show:

```
-- No index on orders
EXPLAIN SELECT order_date, total_amount
FROM orders
WHERE customer_id=42;
 -- Shows: Seq Scan, cost=0.00..450.00

CREATE INDEX idx_cov ON orders(customer_id)
INCLUDE(order_date,total_amount);

EXPLAIN SELECT order_date, total_amount FROM orders WHERE customer_id=42; -
- Now shows: ____?
```

Explanation:

1. PostgreSQL uses the `idx_cov` index to find records matching `customer_id = 42`.
2. The index contains all three required columns:
    1. `customer_id` – filtering column.
    2. `order_date` – selected column.
    3. `total_amount` – selected column.
3. Therefore, PostgreSQL can retrieve the required data directly from the index without accessing the table, provided visibility information allows it.

Comparison:

Before index	After index
Sequential Scan	Index Only Scan
Searches the table	Searches the index
Requires table access	Can avoid table access
More I/O for large tables	Potentially less I/O

Q6) Topic: JSON Queries: JSON_AGG, JSON_EXTRACT & jsonb Operations

Products table has a jsonb column 'attributes' storing:
{"color":"red","weight":1.2,"tags": ["sale", "new"]}

a) Use ->> to extract 'color' as text and -> to extract 'tags' as JSON. Filter to only products where color='red'.

```
SELECT
    id,
    attributes->>'color' AS color,
    attributes->'tags' AS tags
FROM products
WHERE attributes->>'color' = 'red';
```

b) Use JSON_AGG to return each customer_id with a JSON array of all their order IDs and amounts.

```
SELECT
    customer_id,
    JSON_AGG(
        JSON_BUILD_OBJECT(
            'order_id', order_id,
            'amount', total_amount
        )
    ) AS orders
FROM orders
GROUP BY customer_id;
```

c) Use jsonb_set to update the 'weight' attribute of product id=1 from 1.2 to 1.5 without replacing the entire column.

```
UPDATE products
SET attributes = jsonb_set(
    attributes,
    '{weight}',
    '1.5'::jsonb
)
WHERE id = 1;
```

d) Create a GIN index on attributes. Write a query using @> to find all products tagged with 'sale'. Explain why GIN is right for jsonb containment queries.

Create a GIN index:

```sql
CREATE INDEX idx_products_attributes
ON products
USING GIN (attributes);
```

Find products tagged with `sale`:

```sql
SELECT *
FROM products
WHERE attributes @> '{"tags":["sale"]}'::jsonb;
```

Explanation:

1. `@>` checks whether the left JSONB value contains the right JSONB valu
2. The query finds products whose `tags` array contains `"sale"`.

GIN:

1. GIN (Generalized Inverted Index) indexes individual elements inside composite values.
2. It supports efficient JSONB containment searches.
3. It avoids scanning every row when the index is used.

e) Predict the result set: -- products: (id=1, attributes='{"color":"red","weight":1.2,"tags":["sale"]}')

```sql
SELECT id,
    attributes->>'color' AS color,
    (attributes->>'weight')::float + 0.3 AS new_weight,
    attributes->'tags' AS tags FROM products WHERE id=1;
```